

SPRINT 3

TAREA 8

Actualizar una tarea

PROYECTO INDIVIDUAL · PLANIFICADOR DE TAREAS

JavaScript · TaskManager · Web Storage API · JSON · LocalStorage

APERTURA DEL SPRINT 3

En esta última tarea del Sprint 3 continuaremos evolucionando el proyecto Planificador de. El objetivo será persistir las tareas en LocalStorage para que puedan cargarse nuevamente cuando el usuario recargue o vuelva a abrir la página.

La funcionalidad debe integrarse sobre el código existente, conservando los avances, la estructura, el historial de commits y el repositorio del proyecto.

OBJETIVO

Implementar la persistencia de las tareas y del currentId mediante LocalStorage, creando los métodos save() y load() en TaskManager e integrándolos con index.js.

RESULTADO ESPERADO

- TaskManager contiene un método save() que almacena this.tasks y currentId en LocalStorage.
- Las tareas se convierten a JSON mediante JSON.stringify().
- Las tareas almacenadas se recuperan mediante JSON.parse().
- currentId se almacena como String y se convierte nuevamente a Number al cargarlo.
- Después de agregar una tarea y después de actualizar su estado a DONE, se ejecuta taskManager.save().
- Al iniciar la aplicación se ejecutan taskManager.load() y taskManager.render().
- Las tareas permanecen disponibles después de refrescar o volver a abrir la página.
- Las nuevas tareas continúan la secuencia correspondiente de currentId.

ANTES DE COMENZAR

Continúa trabajando sobre el mismo proyecto del Planificador de Tareas. La persistencia debe construirse sobre la estructura y el código desarrollado en la Tarea 7 y en los sprints anteriores.

ANTES	AHORA	RESULTADO
Las tareas existen en this.tasks mientras la página está abierta.	save() guarda this.tasks en LocalStorage.	Las tareas pueden recuperarse después de recargar.
currentId vive en la instancia de TaskManager.	save() guarda currentId.	La numeración puede continuar correctamente.
La aplicación inicia con el estado disponible en memoria.	save() guarda this.tasks en LocalStorage.	Las tareas pueden recuperarse después de recargar.
render() muestra la colección actual	load() se ejecuta antes de render().	La interfaz muestra las tareas persistidas.

ESTRUCTURA QUE DEBES CONSERVAR

```
Planificador-Tareas/  
├── css/  
│   └── styles.css  
├── js/  
│   ├── index.js  
│   └── taskManager.js  
├── index.html  
└── README.md
```

RECUERDA

La estructura puede variar si tu proyecto ya incorpora archivos adicionales. Lo importante es conservar la separación entre HTML, estilos y JavaScript, mantener los avances de los sprints anteriores y realizar esta nueva funcionalidad dentro del mismo repositorio.

DESARROLLO DE LA ACTIVIDAD

PASO 1 - AGREGAR EL MÉTODO `save` A `TaskManager`

En este paso agregarás el método `save()` a `TaskManager` para guardar el arreglo actual `this.tasks` y el `currentId` en `LocalStorage`.

RECURSOS ÚTILES

- Uso de la Web Storage API
- `JSON.stringify()`
- `JSON.parse()`
- String (Cadena)

1. En `js/taskManager.js`, dentro de la clase `TaskManager`, crea un método `save()`. Este método no requiere parámetros.
2. En `save()`, crea un string JSON de las tareas usando `JSON.stringify(this.tasks)` y guárdalo en una variable llamada `tasksJson`.
3. Guarda `tasksJson` en `LocalStorage` bajo la clave `tasks` utilizando `localStorage.setItem()`.
4. Convierte `this.currentId` a `String` y guarda el resultado en una variable llamada `currentId`.
5. Guarda `currentId` en `LocalStorage` bajo la clave `currentId` utilizando `localStorage.setItem()`.
6. En `js/index.js`, después de agregar una nueva tarea y después de actualizar el estado de una tarea a `DONE`, llama a `taskManager.save()`.

```
save() {  
  const tasksJson = JSON.stringify(this.tasks);  
  localStorage.setItem('tasks', tasksJson);  
  
  const currentId = String(this.currentId);  
  localStorage.setItem('currentId', currentId);  
}
```

PRUEBA EL PASO 1

1. Abre index.html en el navegador y crea una nueva tarea usando el formulario.
2. Abre las herramientas de desarrollador y ve a la pestaña Application.
3. En la barra lateral, bajo Storage, expande Local Storage y selecciona file://.
4. Verifica que exista una clave tasks con el arreglo de tareas convertido a string.
5. Verifica que exista una clave currentId con el valor actual de currentId.

Resultado esperado: deberías ver las claves tasks y currentId almacenadas en Local Storage.

PASO 2 · AGREGAR EL MÉTODO load A TaskManager

Ahora que guardamos nuestras tareas en LocalStorage, necesitamos una forma de cargarlas de vuelta en TaskManager cuando se carga la aplicación. De igual manera, debemos cargar nuevamente currentId.

RECURSOS ÚTILES

- Uso de la Web Storage API
 - JSON.parse()
 - Number()
1. En js/taskManager.js, agrega un nuevo método llamado load(). Este método no requiere parámetros.
 2. En load(), revisa si hay tareas guardadas en LocalStorage con localStorage.getItem().
 3. Si hay tareas guardadas, obtén el string JSON con localStorage.getItem('tasks') y guárdalo en una variable llamada tasksJson.
 4. Convierte tasksJson a un arreglo usando JSON.parse() y guárdalo en this.tasks.
 5. Revisa si currentId está guardado en LocalStorage con localStorage.getItem().
 6. Si currentId está guardado, obtén su valor y guárdalo en una variable llamada currentId.
 7. Convierte currentId a un número antes de guardarlo en this.currentId.
 8. En js/index.js, cerca del inicio del archivo, después de instanciar taskManager, carga las tareas con taskManager.load() y luego renderízalas con taskManager.render().

```
load() {
  const tasksJson = localStorage.getItem('tasks');

  if (tasksJson) {
    this.tasks = JSON.parse(tasksJson);
  }

  const currentId = localStorage.getItem('currentId');

  if (currentId) {
    this.currentId = Number(currentId);
  }
}
```

INTEGRAR load() EN index.js

```
const taskManager = new TaskManager();

taskManager.load();
taskManager.render();
```

PRUEBA EL PASO 2

6. Abre index.html y agrega una tarea.
7. Recarga la página.
8. Comprueba que la tarea creada anteriormente vuelve a aparecer.

Crea una nueva tarea y verifica que utilice el siguiente currentId después del valor guardado

FLUJO DE LA PERSISTENCIA

CREAR / ACTUALIZAR TAREA



TaskManager modifica this.tasks



taskManager.save()



JSON.stringify(this.tasks)



localStorage.setItem('tasks', ...)



localStorage.setItem('currentId', ...)

AL CARGAR LA APLICACIÓN



taskManager.load()



localStorage.getItem('tasks')



JSON.parse(tasksJson)



this.tasks recupera las tareas



localStorage.getItem('currentId')



Number(currentId)



this.currentId recupera el identificador



taskManager.render()



La interfaz muestra las tareas persistidas

¿POR QUÉ JSON?

LocalStorage solo puede almacenar valores de tipo String. Por eso debemos convertir el arreglo de tareas a una cadena mediante `JSON.stringify()` antes de guardarlo y convertirlo nuevamente a un arreglo mediante `JSON.parse()` al cargarlo.

PRUEBAS DE FUNCIONALIDAD

No te limites a comprobar únicamente que una tarea se guarda. Verifica que la persistencia funcione después de recargar la página, que el estado actualizado se conserve y que `currentId` permita continuar creando tareas sin perder la secuencia.

CASO 1 · GUARDAR UNA TAREA

- ✓ Abre la aplicación.
- ✓ Crea una nueva tarea.
- ✓ Comprueba que la tarea aparece en la lista.
- ✓ Verifica en Application → Local Storage → file:// que existan tasks y currentId.

CASO 2 · RECUPERAR TAREAS DESPUÉS DE RECARGAR

- ✓ Crea al menos dos tareas.
- ✓ Recarga la página.
- ✓ Comprueba que las tareas vuelven a aparecer.
- ✓ Verifica que la información de cada tarea se mantiene.

CASO 3 · CONSERVAR EL ESTADO DONE

- ✓ Crea al menos dos tareas.
- ✓ Marca una tarea como DONE.
- ✓ Recarga la página.
- ✓ Comprueba que la tarea continúa mostrando el estado actualizado.

CASO 4 · VERIFICAR `currentId`

- ✓ Crea varias tareas y observa sus identificadores.
- ✓ Recarga la página.
- ✓ Crea una nueva tarea.
- ✓ Comprueba que el nuevo identificador continúa después del `currentId` guardado y no reinicia la numeración.

CASO 5 · CONSERVAR LOS AVANCES ANTERIORES

- ✓ Comprueba que puedes crear tareas.
- ✓ Comprueba que puedes marcar una tarea como DONE.
- ✓ Comprueba que puedes eliminar una tarea si esa funcionalidad ya fue implementada.
- ✓ Recarga la página y verifica que las funcionalidades y datos persistidos continúan funcionando.

LISTA DE VERIFICACIÓN

☐	TaskManager contiene el método save().
☐	save() convierte this.tasks utilizando JSON.stringify().
☐	Las tareas se guardan con la clave tasks mediante localStorage.setItem().
☐	this.currentId se convierte a String.
☐	currentId se guarda mediante localStorage.setItem().
☐	TaskManager contiene el método load().
☐	load() consulta LocalStorage mediante localStorage.getItem().
☐	tasks se recupera utilizando la clave tasks.
☐	tasksJson se convierte nuevamente a arreglo mediante JSON.parse().
☐	currentId se recupera utilizando la clave currentId.
☐	currentId se convierte nuevamente a Number.
☐	Después de agregar una tarea se ejecuta taskManager.save().

ENTREGA · CIERRE DEL SPRINT 3

Esta es la última entrega del Sprint 3. La revisión se realizará sobre el repositorio completo para validar los avances construidos durante las tareas del sprint y comprobar la integración de la persistencia.

- Continúa utilizando el mismo repositorio del proyecto.
- Conserva el historial de commits y los avances desarrollados durante el Sprint 3.
- Verifica que el README esté actualizado.
- El enlace al deploy debe estar actualizado en el README, tal como se solicitó en el cierre del Sprint 1.
- El deploy debe corresponder a la versión final del proyecto y permitir comprobar las funcionalidades implementadas.
- Antes de entregar, comprueba que crear, actualizar, eliminar (si aplica) y persistir tareas funcionen correctamente.
- La entrega debe cumplir con los criterios definidos en la Rúbrica de Evaluación.